End-Term Report

Summer of Science 2026
Large Language Models

GeoQuery: An AI-Powered Natural Language GIS Query System

Jāgu Manish
Roll No.: 25M0612
M. Tech (Remote Sensing)
Civil Engineering Department
IIT Bombay

Table of Contents

1 Introduction

Geographic Information Systems (GIS) have become an essential technology for managing, analyzing, and visualizing spatial data across diverse application domains, including urban planning, transportation, environmental monitoring, disaster management, and public infrastructure. Modern GIS platforms enable users to perform complex spatial analyses and retrieve geospatial information from large datasets. However, most GIS software requires users to possess substantial technical knowledge, including familiarity with spatial databases, query languages such as SQL, and specialized GIS operations. This technical barrier often limits the accessibility of geospatial information to non-expert users.

Recent advancements in Artificial Intelligence (AI), particularly in Large Language Models (LLMs), have introduced new possibilities for human-computer interaction through natural language. LLMs can understand user intent expressed in everyday language and convert it into structured information suitable for computational processing. These models have demonstrated remarkable performance across a range of natural language processing tasks, including question answering, information retrieval, code generation, and conversational assistance (Brown et al., 2020). Their ability to interpret complex user requests makes them a promising technology for simplifying interactions with GIS applications.

Natural Language Interfaces (NLIs) for GIS aim to bridge the gap between natural language and spatial databases, enabling users to retrieve geographic information with simple textual queries. Instead of manually constructing SQL statements or navigating multiple GIS tools, users can express their requirements in natural language, enabling more intuitive access to geospatial data. Recent developments in LLMs have significantly improved the capability of such interfaces by accurately identifying user intent, extracting spatial entities, and generating structured outputs that can be translated into executable database queries. This integration could make GIS systems more accessible to users without prior spatial analysis expertise.

GeoQuery has been developed with this objective in mind. It is an AI-powered Natural Language GIS Query System that enables users to retrieve and visualize geospatial information through conversational queries. The system integrates a Large Language Model for query planning, PostgreSQL with the PostGIS extension for spatial data management, and a Streamlit-based interactive interface for visualization. User queries are first interpreted by the language model, converted into structured JSON representations, translated into spatial SQL queries, executed on a PostGIS database, and finally displayed as interactive maps and attribute tables. The system supports the retrieval of multiple geospatial layers, automatic identification of administrative boundaries, spatial filtering, visualization of query results, and export of retrieved datasets.

The primary objective of this project is to simplify geospatial information retrieval by eliminating the need for users to possess detailed knowledge of GIS software or SQL

syntax. By combining recent advances in Large Language Models with established spatial database technologies, GeoQuery provides an intelligent and user-friendly interface for querying geospatial datasets. The system demonstrates how AI can enhance GIS accessibility while maintaining the robustness and efficiency of spatial database operations. This report presents the design, implementation, and evaluation of the GeoQuery system, including its architecture, workflow, database design, query planning mechanism, visualization components, and experimental results.

2 GIS Fundamentals and Technologies Used

2.1 Geographic Information Systems (GIS)

A Geographic Information System (GIS) is a computer-based system designed for capturing, storing, managing, analyzing, and visualizing geographically referenced data. Unlike conventional information systems that primarily deal with textual or numerical information, GIS integrates spatial and attribute information to represent real-world entities and their relationships. By combining location-based information with descriptive attributes, GIS enables users to perform spatial analysis, identify geographic patterns, and support informed decision-making across a wide range of disciplines.

The importance of GIS has grown significantly over the past few decades due to the increasing availability of geospatial data from satellites, sensors, drones, GPS devices, and volunteered geographic information platforms such as OpenStreetMap. Governments, industries, and research organizations extensively use GIS for urban planning, transportation management, disaster risk assessment, environmental monitoring, precision agriculture, natural resource management, healthcare planning, and infrastructure development.

A GIS consists of five major components:

- **Hardware:** Computers, servers, GPS devices, and networking infrastructure used for data storage and processing.
- **Software:** GIS applications such as ArcGIS Pro, QGIS, PostGIS, and web mapping frameworks that enable spatial analysis and visualization.
- **Data:** Spatial and non-spatial information representing geographic entities.
- **People:** Analysts, planners, researchers, and decision-makers who utilize GIS for solving real-world problems.
- **Methods:** Standardized workflows and analytical procedures used for spatial data management and analysis.

The strength of GIS lies in its capability to answer spatial questions such as:

- What features are located at a particular location?
- Which hospitals are located within a district?
- What is the nearest emergency facility?
- Which rivers intersect a protected area?
- What is the shortest route between two locations?

These spatial queries distinguish GIS from conventional database management systems.

GeoQuery has been developed around these core GIS principles by enabling users to retrieve geographic information through natural language rather than manually writing SQL queries or using traditional GIS software interfaces.

2.2 Spatial Data Models

GIS represents geographic information using two primary data models: vector and raster.

2.2.1 Vector Data Model

The vector data model represents discrete geographic features using geometric primitives.

- **Points** represent individual locations such as hospitals, schools, ATMs, bus stops, and railway stations.
- **Lines** represent linear features including roads, rivers, canals, railways, and pipelines.
- **Polygons** represent areal features such as administrative boundaries, lakes, forests, land parcels, and protected areas.

Each spatial feature is associated with an attribute table containing descriptive information such as feature name, category, identification number, and other thematic attributes.

GeoQuery primarily operates on vector datasets because vector data is well suited for spatial querying, filtering, and visualization.

2.2.2 Raster Data Model

Raster data represents continuous geographic phenomena using regularly spaced grid cells (pixels). Each pixel stores a numerical value corresponding to a particular physical property such as elevation, rainfall, temperature, vegetation index, or satellite reflectance.

Raster datasets are widely used in remote sensing and environmental modelling. Examples include:

- Digital Elevation Models (DEM)
- Land Surface Temperature

- NDVI
- Rainfall
- Satellite imagery

Although raster data is fundamental to GIS, the current implementation of GeoQuery primarily focuses on vector datasets from OpenStreetMap.

2.3 Spatial Queries and Spatial Relationships

Unlike conventional databases, GIS databases support spatial relationships between geographic features. These relationships enable advanced geographic analysis beyond attribute-based filtering.

Common spatial relationships include:

- **Intersection:** Determines whether two geometries overlap or cross each other.
- **Containment:** Determines whether one geometry completely contains another.
- **Within:** Identifies features located inside another geographic region.
- **Touches:** Identifies geometries sharing common boundaries.
- **Distance:** Measures the spatial distance between geographic features.
- **Nearest Neighbor:** Finds the closest feature to a specified location.

GeoQuery extensively uses spatial relationships via PostGIS functions to retrieve geographically relevant information from user queries.

2.4 PostgreSQL

PostgreSQL is an advanced open-source object-relational database management system known for its reliability, extensibility, and standards compliance. It supports advanced indexing, transaction management, concurrency control, and large-scale data storage.

Compared with traditional relational databases, PostgreSQL provides several advantages:

- ACID-compliant transactions
- High reliability and fault tolerance
- Advanced indexing techniques

- User-defined functions
- Excellent scalability
- Open-source architecture

These characteristics make PostgreSQL particularly suitable for handling large geospatial datasets.

In GeoQuery, PostgreSQL serves as the primary database for storing OpenStreetMap-derived vector datasets.

2.5 PostGIS

PostGIS is an open-source spatial extension for PostgreSQL that transforms the database into a full-featured spatial database management system.

PostGIS introduces support for geographic data types including:

- Point
- MultiPoint
- LineString
- MultiLineString
- Polygon
- MultiPolygon

It also provides hundreds of spatial functions for geographic analysis.

Examples include,

Function	Purpose
ST_Intersects()	Tests whether two geometries intersect
ST_Within()	Determines whether a geometry lies within another
ST_Contains()	Checks containment relationship
ST_Buffer()	Creates buffer regions
ST_DWithin()	Finds nearby features within a specified distance
ST_Distance()	Calculates the distance between geometries

Function	Purpose
ST_Area()	Computes polygon area
ST_Length()	Calculates line length
ST_Centroid()	Determines the geometric center
ST_Union()	Merges multiple geometries

GeoQuery relies extensively on PostGIS for executing spatial SQL queries generated from natural language input. Spatial indexing using GiST indexes further improves query performance for large datasets.

2.6 OpenStreetMap and Geofabrik

OpenStreetMap (OSM) is a collaborative, open-source mapping project that provides freely accessible geographic data contributed by volunteers worldwide. The database contains detailed information on transportation networks, buildings, administrative boundaries, land use, water bodies, points of interest, and numerous other geographic features.

Geofabrik provides regularly updated extracts of OpenStreetMap data in GIS-compatible formats, enabling efficient download of country- or region-specific datasets.

For this project, India-wide Geofabrik datasets were imported into PostgreSQL/PostGIS. The database includes thematic layers such as:

- Points of Interest (POI)
- Waterways
- Water Bodies
- Natural Features
- Transportation Networks
- Railways
- Land Use
- Administrative Boundaries (States, Districts, Subdistricts)

These layers form the primary data source used by GeoQuery.

2.7 Large Language Models (LLMs)

Large Language Models are deep neural networks trained on massive text corpora to understand and generate human language. Their ability to interpret user intent makes them particularly useful for Natural Language Interfaces.

GeoQuery integrates the **Llama 3.3 70B Versatile** model through the **Groq API**. Rather than generating SQL queries directly, the language model converts user questions into structured JSON representations that include the target layer, feature class, location, and query parameters. This intermediate representation improves reliability, modularity, and system security by separating language understanding from SQL generation.

2.8 Streamlit

Streamlit is an open-source Python framework for developing interactive web applications for data science and machine learning. It provides a simple programming interface for rapidly building dashboards and visualization tools.

In GeoQuery, Streamlit provides the graphical user interface through which users submit natural language queries, view retrieved spatial information, inspect query statistics, and download results.

2.9 Folium

Folium is a Python library built on the Leaflet JavaScript mapping framework for generating interactive web maps.

GeoQuery employs Folium to visualize retrieved spatial features using multiple basemaps, including OpenStreetMap and Google Satellite imagery. Additional functionalities such as full-screen mode, measurement tools, layer controls, automatic zooming, and administrative boundary visualization enhance the user experience.

2.10 Geopandas

GeoPandas extends the Pandas library to support spatial vector data. It enables efficient reading, writing, manipulation, and visualization of geospatial datasets.

Within GeoQuery, GeoPandas retrieves spatial query results directly from the PostGIS database, processes geometry objects, prepares attribute tables for display, and facilitates exporting query results into GIS-compatible formats.

2.11 Technology Stack of GeoQuery

The GeoQuery system integrates multiple open-source technologies to create an intelligent Natural Language GIS Query System. User queries entered through the Streamlit interface are interpreted by the Llama 3.3 language model via the Groq API. The structured query is translated into spatial SQL, executed on a PostgreSQL database enhanced with PostGIS, and the resulting geometries are processed using GeoPandas. Finally, Folium renders spatial features on an interactive map alongside administrative boundaries and attribute information, enabling users to explore geographic information through a simple conversational interface.

3 Data Preparation and Spatial Database Development

3.1 Overview

The performance and reliability of any Geographic Information System largely depend on the quality and organization of its spatial database. Since GeoQuery retrieves spatial information directly from a PostgreSQL/PostGIS database, considerable effort was devoted to collecting, preparing, validating, and organizing geospatial datasets before developing the application. This chapter describes the complete workflow adopted for preparing the spatial database, including data acquisition, preprocessing, administrative boundary preparation, database creation, and spatial indexing.

The objective of this stage was to create a centralized spatial database containing multiple thematic layers covering the entire Indian subcontinent. The prepared database serves as the foundation for all spatial queries performed by GeoQuery.

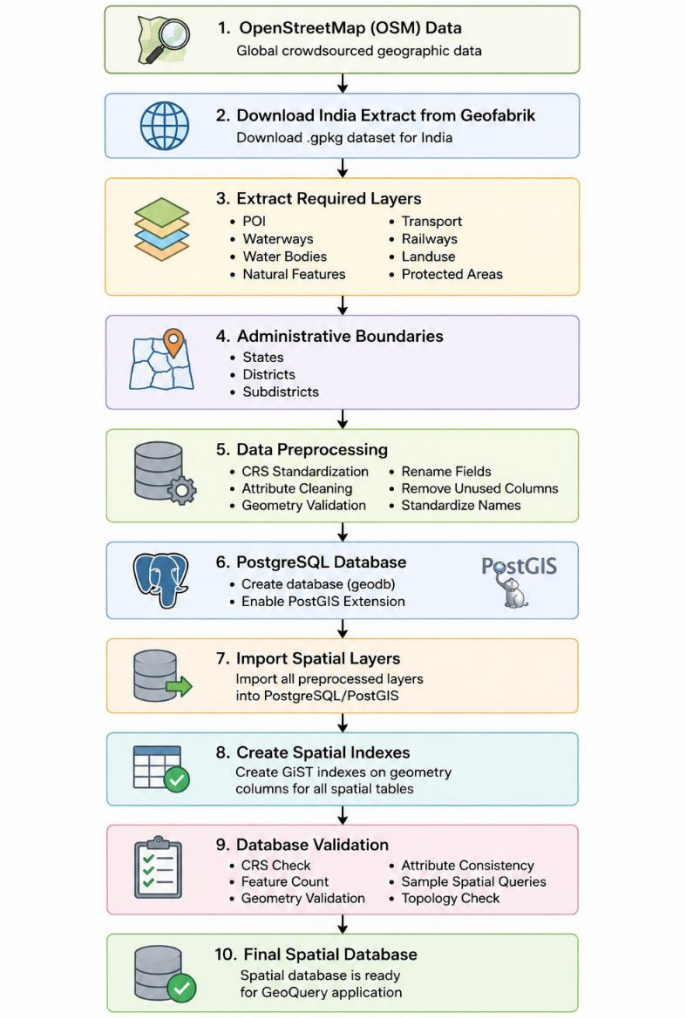

Figure 3.1: Data Preparation Workflow

3.2 Data Acquisition

The primary source of spatial data used in this project is OpenStreetMap (OSM). OpenStreetMap is an open-source, collaborative mapping platform that provides freely accessible, continuously updated geographic information contributed by volunteers worldwide. It contains comprehensive information regarding transportation networks, administrative boundaries, land use, natural features, water bodies, and numerous points of interest.

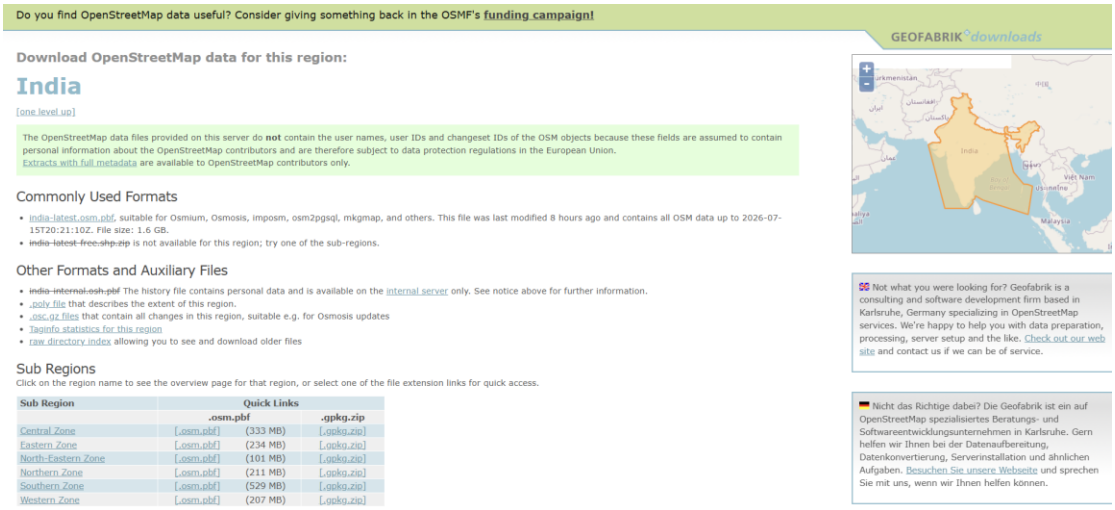

Figure 3.2 The Geofabrik India dataset download page.

To obtain India-wide datasets in a GIS-compatible format, the Geofabrik data distribution service was used. Geofabrik periodically extracts OpenStreetMap data and distributes it in various GIS formats, including Shapefile, Geopackage, and Protocol Buffer Binary Format (PBF). The India extract was downloaded in Geopackage (.gpkg) format, which allows multiple spatial layers to be stored efficiently within a single file.

The downloaded dataset contains numerous thematic layers representing different categories of geographic features. Only the layers relevant to the GeoQuery application were selected for import into the spatial database. The primary layers used in this project are listed in Table 3.1

Table 3.1 Spatial Layers Used in GeoQuery

Layer	**Geometry Type**	**Description**
POI	Point	Hospitals, schools, banks, ATMs, restaurants, etc.
Places	Point	Cities, towns, villages, and settlements
Waterways	Line	Rivers, streams and canals

Water	Polygon	Lakes, reservoirs and ponds
Railways	Line	Railway lines and stations
Transport	Line	Road transportation network
TransportP	Point	Airports, bus stations and transport facilities
Natural	Polygon	Forests, hills and natural features
Landuse	Polygon	Residential, industrial and agricultural land
Protected Areas	Polygon	Wildlife sanctuaries and national parks

These datasets constitute the primary thematic layers utilized by GeoQuery for spatial information retrieval.

3.3 Administrative Boundary Preparation

Unlike thematic datasets obtained from OpenStreetMap, administrative boundaries require a separate preparation process because they serve as reference layers for spatial filtering. GeoQuery supports queries at multiple administrative levels including states, districts, and subdistricts. Therefore, accurate administrative boundary datasets were prepared before database creation.

The administrative boundary datasets were obtained from publicly available government and open-source repositories. The collected datasets initially contained numerous descriptive attributes and inconsistent field names. Since GeoQuery identifies administrative regions using natural language, the datasets were standardized by retaining only the necessary fields and creating a common attribute named "name" across all administrative levels.

The preparation process involved:

- Verification of coordinate reference systems.
- Reprojection of all boundary datasets to EPSG:4326 (WGS84).
- Removal of redundant attribute fields.
- Standardization of administrative names.
- Validation of polygon geometries.
- Harmonization of attribute schemas.

Three administrative layers were finally prepared:

- State Boundaries
- District Boundaries
- Subdistrict Boundaries

These layers are later used by the location resolution module to automatically determine the appropriate administrative level during query execution.

3.4 Data Preprocessing

The downloaded datasets required several preprocessing operations before they could be imported into PostgreSQL. Since OpenStreetMap contains many attributes not required by the present application, unnecessary fields were removed to reduce storage requirements and improve query performance.

The preprocessing workflow consisted of the following steps:

- Inspection of available layers within the Geopackage.
- Selection of thematic layers required for the application.
- Removal of unnecessary attribute fields.
- Verification and correction of geometry types.
- Reprojection of datasets to the common coordinate system (EPSG:4326).
- Validation of geometry integrity.
- Standardization of attribute names.
- Export of cleaned datasets for database import.

These preprocessing steps ensured consistency across all datasets and simplified the subsequent SQL query generation process.

3.5 Spatial Database Creation

The backend database of GeoQuery was developed using PostgreSQL together with the PostGIS spatial extension. PostgreSQL provides a robust relational database management system, while PostGIS extends PostgreSQL by introducing spatial data types, indexing mechanisms, and geographic analysis functions.

Initially, a new PostgreSQL database named geodb was created. The PostGIS extension was then enabled to support the storage and manipulation of spatial geometries.

The database was initialized using the following SQL commands:

```
CREATE DATABASE geodb;

\c geodb;

CREATE EXTENSION postgis;
```

Enabling the PostGIS extension allows PostgreSQL to store geometry objects and perform spatial operations such as intersection, containment, buffering, distance calculation, and area estimation.

3.6 Importing Spatial Data into PostgreSQL

After preprocessing, the cleaned datasets were imported into the PostgreSQL/PostGIS database. The import process was carried out individually for each thematic layer to maintain a modular database structure.

Each imported layer was stored as an independent PostgreSQL table while preserving its geometry information. During import, PostgreSQL automatically recognized the geometry column and stored it using the PostGIS Geometry data type.

The imported database contains thematic tables representing various categories of geographic information, allowing efficient retrieval of individual datasets without unnecessary joins.

The final database structure includes:

- poi
- places
- waterways
- water
- railways
- transport
- transportp (Transport dataset as point data)
- natural
- landuse

- protectedareas
- states
- districts
- subdistricts

This modular organization improves maintainability and simplifies query generation.

3.7 Database Schema

This modular organization improves maintainability and simplifies query generation

Each spatial table contains two major categories of information:

- Attribute Information
- Spatial Geometry

The attribute information includes descriptive fields such as feature name, feature class, OpenStreetMap identifier, and other thematic attributes.

The geometry column stores the spatial representation of each geographic feature using PostGIS geometry objects. A simplified representation of the database schema is shown in Table 3.2.

Table 3.2 Database Schema

Column	**Description**
osm_id	OpenStreetMap identifier
name	Feature name
fclass	Feature category
code	Classification code
geom	Geometry object

The geom column serves as the basis for all spatial operations performed in GeoQuery.

3.8 Spatial Indexing

Spatial queries often involve searching among millions of geographic features. Without indexing, every spatial query would require a full table scan, resulting in poor performance.

To improve query efficiency, GiST (Generalized Search Tree) spatial indexes were created on the geometry column of each spatial table.

A typical spatial index is created using:

```
CREATE INDEX idx_poi_geom
ON poi
USING GIST (geom);
```

Similar indexes were created for all thematic layers and administrative boundary tables.

Spatial indexes significantly reduce query execution time by allowing PostgreSQL to quickly identify candidate geometries before performing computationally expensive spatial operations.

3.9 Database Validation

After importing the datasets, several validation procedures were performed to ensure database correctness.

The validation process included:

- Verification of feature counts.
- Checking coordinate reference systems.
- Validation of geometry integrity.
- Testing attribute consistency.
- Verification of administrative boundary names.
- Execution of sample spatial SQL queries.

Several PostGIS functions including ST_Intersects(), ST_Area(), ST_Length(), and ST_Centroid() were executed to verify that the imported geometries were stored correctly and spatial operations produced valid results.

Successful completion of these validation steps confirmed that the database was ready for integration with the GeoQuery application.

3.10 Final Spatial Database

The completed PostgreSQL/PostGIS database serves as the central repository for all

spatial information used by GeoQuery. It integrates multiple thematic layers with administrative boundaries, enabling efficient execution of natural-language GIS queries.

The final database consists of thirteen spatial tables containing vector data representing points, lines, and polygons across the Indian subcontinent. These datasets provide comprehensive geographic coverage while enabling rapid spatial queries via PostGIS indexing and optimized SQL queries.

The prepared spatial database serves as the foundation for the GeoQuery application's natural language understanding, spatial query generation, geographic visualization, and data retrieval.

4 System Architecture

4.1 Overview

GeoQuery has been designed as a modular Natural Language Geographic Information System that integrates artificial intelligence with spatial database technologies. The primary objective of the system is to enable users to retrieve and visualize geospatial information using natural language instead of manually constructing SQL queries or using conventional GIS software interfaces.

The system consists of six major components: the user interface, natural language query planner, query parser, spatial query generator, spatial database, and visualization module. These components work together to convert a user's natural language request into executable spatial SQL and display the retrieved geographic features on an interactive map.

The modular architecture provides flexibility, maintainability, and scalability by separating the responsibilities of language understanding, query generation, database execution, and visualization into independent modules.

4.2 Overall System Architecture

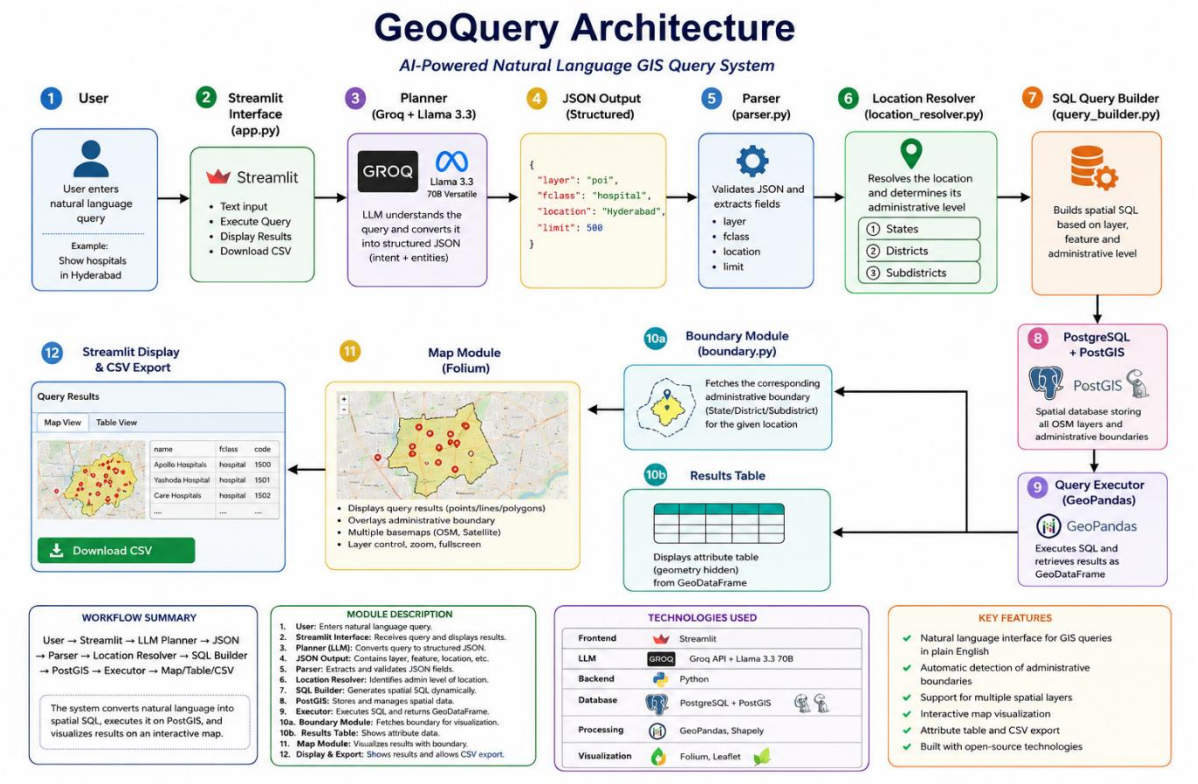

Figure 4.1 illustrates the complete workflow of the GeoQuery system.

4.3 User Interface Module

The user interface has been developed using the Streamlit framework to provide a lightweight and interactive environment for querying geospatial datasets. The interface consists of a natural language input box, execution controls, an interactive map, a results table, and an information panel.

Users submit their geographic queries in plain English, such as *"Show hospitals in Hyderabad"* or *"Show rivers in Telangana"*. The interface sends the user query to the planning module and displays the retrieved spatial data upon successful execution. The application also allows users to download the retrieved attributes as CSV files for further analysis.

The interface automatically updates the map extent based on the retrieved spatial features and displays relevant administrative boundaries for improved spatial context.

4.4 Query Planner Module

The query planner forms the intelligence layer of the GeoQuery system. It utilizes the **Llama 3.3 70B Versatile** Large Language Model through the Groq API to interpret user queries.

Instead of generating SQL statements directly, the planner converts natural language into a structured JSON representation. This intermediate representation improves modularity and reduces the possibility of generating unsafe SQL queries.

For example,

User Query:

Show hospitals in Hyderabad

is converted into

JSON:

```
{
   "layer":"poi",
   "fclass":"hospital",
   "location":"Hyderabad",
   "limit":500
}
```

The planner identifies the requested feature class, spatial layer, location, and query constraints without requiring the user to understand GIS terminology or SQL syntax.

4.5 Query Parsing and Validation

The parser validates the JSON output generated by the planner and extracts the required query parameters. These parameters include the spatial layer, feature class, location, result limit, and other optional constraints.

The parser also performs basic preprocessing to ensure that the generated plan conforms to the expected schema before forwarding it to the SQL generation module.

This separation between planning and query generation improves system reliability and simplifies debugging.

4.6 Location Resolution Module

One challenge in natural language GIS queries is determining a location's administrative level. A location name may correspond to a state, district, or subdistrict.

Instead of relying solely on the language model, GeoQuery incorporates a database-driven location resolver. The resolver searches the administrative boundary tables in PostGIS and automatically determines whether the specified location falls within the States, Districts, or Subdistricts layer.

This approach increases system reliability by using authoritative spatial data rather than relying solely on language model predictions.

4.7 SQL Query Generation

The SQL Query Builder converts the parsed JSON representation into executable spatial SQL statements.

The module dynamically generates SQL based on the requested spatial layer, feature class, administrative boundary, and user-defined constraints. Spatial filtering is performed using PostGIS functions such as **ST_Intersects()**, enabling efficient retrieval of geographic features within the specified administrative region.

The generated SQL is subsequently executed on the PostgreSQL/PostGIS database.

4.8 Spatial Database

GeoQuery utilizes PostgreSQL with the PostGIS extension as the backend spatial database.

The database contains multiple OpenStreetMap-derived thematic layers imported from Geofabrik, including:

- Points of Interest
- Waterways
- Railways
- Transportation
- Land Use
- Natural Features
- Water Bodies
- Administrative Boundaries

Spatial indexes have been created to improve query performance during spatial filtering and intersection operations.

4.9 Query Execution

The execution module submits the generated SQL query to PostgreSQL and retrieves the resulting geometries using GeoPandas.

The execution time is recorded for performance evaluation, while the retrieved spatial features are stored as GeoDataFrames for further visualization and export.

The module also handles empty result sets and database exceptions, ensuring robust execution of user queries.

4.10 Visualization Module

The visualization module has been developed using Folium.

Retrieved geometries are rendered on an interactive web map using appropriate symbology based on geometry type.

Point features are displayed with circular markers and informative pop-ups, while polygon and line features are rendered as GeoJSON layers. Administrative boundaries corresponding

to the query location are highlighted to provide geographic context.

Additional visualization functionalities include:

- OpenStreetMap basemap
- Google Satellite imagery
- Layer control
- Fullscreen mode
- Measurement tool
- Automatic map zooming
- Interactive feature popups

4.11 Overall Workflow

The operational workflow of GeoQuery consists of the following sequential steps:

1. The user submits a natural language GIS query through the Streamlit interface.
2. The query is interpreted by the Large Language Model, which generates a structured JSON representation.
3. The parser validates and extracts query parameters.
4. The location resolver identifies the corresponding administrative boundary.
5. The SQL Query Builder constructs an executable spatial SQL query.
6. PostgreSQL/PostGIS executes the query.
7. GeoPandas retrieves the resulting geometries.
8. Folium visualizes the retrieved spatial features on an interactive map.
9. The attribute table, execution statistics, and downloadable results are presented to the user.

This modular workflow enables seamless integration between artificial intelligence, spatial databases, and web-based GIS visualization while maintaining a clear separation between language understanding, spatial query generation, and data visualization.

5 System Implementation

5.1 Overview

This chapter presents the implementation details of the GeoQuery system. The application was developed using Python and integrates Large Language Models (LLMs), PostgreSQL/PostGIS, and web-based GIS visualization to provide an intelligent natural language interface for querying geospatial data. The implementation consists of several independent modules responsible for user interaction, natural language understanding, query generation, database execution, and visualization. A modular design was adopted to improve maintainability, simplify debugging, and allow future expansion of the system.

5.2 Development Environment

The GeoQuery application was developed using open-source software and libraries. Python was used as the primary programming language due to its extensive support for geospatial analysis, database connectivity, and artificial intelligence. PostgreSQL with the PostGIS extension was employed as the backend spatial database for storing and querying geographic information. The Streamlit framework was used to develop the web-based graphical interface, while Folium was utilized for interactive map visualization. The Groq API, powered by the Llama 3.3 70B Versatile model, was integrated to interpret natural-language user queries.

Table 5.1 Development Environment

Component	Purpose
Python	Application development
PostgreSQL	Spatial database
PostGIS	Spatial SQL operations
Streamlit	Web interface
Folium	Interactive maps
GeoPandas	Spatial data processing
SQLAlchemy	Database connectivity
Groq API	Large Language Model inference
Llama 3.3 70B Versatile	Natural language understanding
Visual Studio Code	Development environment

5.3 Project Structure

The GeoQuery application follows a modular architecture in which each module performs a specific task within the processing pipeline. This separation of responsibilities improves readability and facilitates future modifications without affecting other components of the system. The overall project structure is illustrated in Fig. 5 .1. The purpose of each module is summarized in Table 5.2.

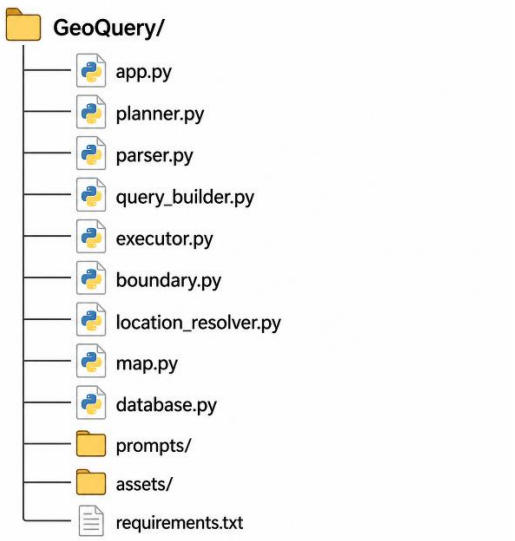

Figure 5.1 GeoQuery directory structure

Table 5.2 Purpose of modules

Module	**Description**
app.py	Main Streamlit application
planner.py	Sends user query to LLM
parser.py	Parses JSON response
query_builder.py	Generates SQL query
executor.py	Executes SQL on PostgreSQL
boundary.py	Retrieves the administrative boundary
location_resolver.py	Determines location type
map.py	Displays an interactive map
database.py	Database connection

5.4 User Interface

The user interface was developed with Streamlit to provide a lightweight, interactive environment for querying spatial data. The interface consists of a natural language input box, query execution controls, an interactive map, a results table, and an information panel displaying execution statistics.

Users enter geographic questions in plain English, such as *"Show hospitals in Hyderabad"* or *"Show rivers in Telangana"*. Upon execution, the application automatically processes the request, retrieves the corresponding geographic features, and visualizes them on the map. Users can also download the retrieved attribute information as a CSV file for further analysis.

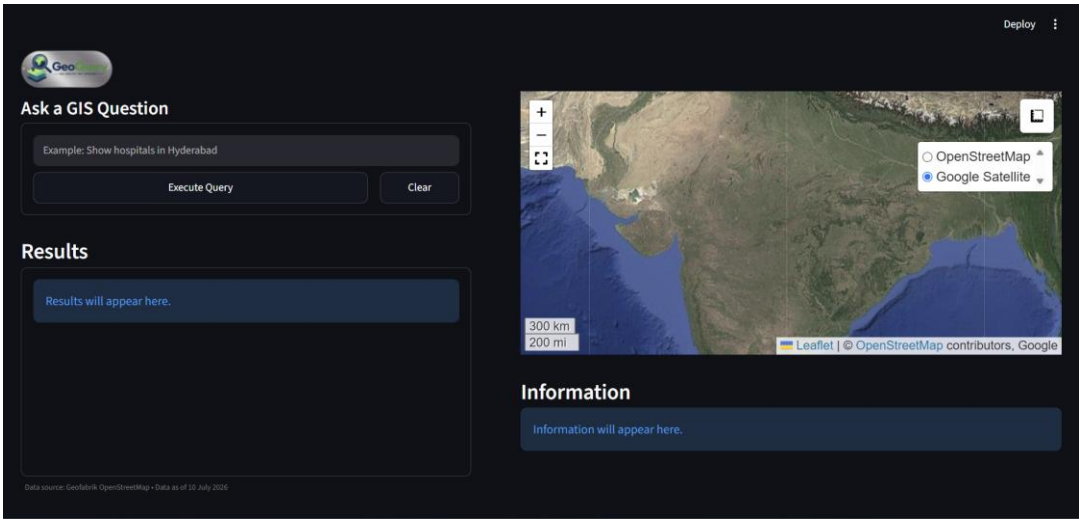

Figure 5.2: GeoQuery User Interface

5.5 Natural Language Query Processing

Natural language understanding is performed using the Llama 3.3 70B Versatile Large Language Model accessed through the Groq API. Instead of generating SQL directly, the language model converts the user's request into a structured JSON representation. This intermediate representation separates language understanding from SQL generation and improves system reliability.

For example, the user query

Show hospitals in Hyderabad

is converted into

```
{
  "layer":"poi",
```

```
    "fclass":"hospital",
    "location":"Hyderabad",
    "limit":500
}
```

The planner identifies the requested thematic layer, feature class, administrative location, and result limit. The generated JSON is then forwarded to the parser for further processing.

5.6 Query Parsing

The parser validates the JSON generated by the language model before it is used for SQL generation. It ensures that all mandatory fields are present and extracts relevant parameters such as the target layer, feature class, administrative location, and query limit.

The parser also performs basic error checking and assigns default values when the language model omits optional fields. This validation stage prevents malformed queries from reaching the database layer.

5.7 Location Resolution

Administrative locations mentioned in user queries may correspond to states, districts, or subdistricts. To accurately determine the appropriate administrative level, GeoQuery includes a location-resolution module that searches the administrative boundary tables stored in PostgreSQL.

Instead of depending entirely on the language model, the resolver queries the States, Districts, and Subdistricts tables sequentially until a matching location is identified. This approach significantly improves the reliability of administrative boundary selection.

For example, when the user specifies "Hyderabad", the resolver identifies it as a district, whereas "Telangana" is identified as a state. The resolved administrative level determines which boundary table is used during SQL generation.

5.8 SQL Query Generation

The SQL Query Builder converts the parsed JSON representation into executable PostGIS SQL statements. The generated query is constructed dynamically based on the

requested layer, feature class, administrative location, and optional constraints.

For administrative filtering, the corresponding boundary table is joined using the PostGIS spatial predicate ST_Intersects(). Attribute filtering is performed using SQL WHERE clauses generated from the parsed parameters.

A simplified example is shown below.

```
SELECT poi.*
FROM poi
JOIN districts d
ON ST_Intersects(
   poi.geom,
   d.geom
)
WHERE
LOWER(d.name)=LOWER('Hyderabad')
AND
LOWER(poi.fclass)=LOWER('hospital')
LIMIT 500;
```

The modular design of the SQL builder allows the same implementation to support different spatial layers and administrative levels without requiring separate query templates.

5.9 Query Execution

The generated SQL query is executed using PostgreSQL through SQLAlchemy, while GeoPandas retrieves the resulting geometries directly as GeoDataFrames.

The execution module records the query execution time, counts the number of retrieved features, determines the geometry type, and prepares the results for visualization. Error-handling mechanisms are in place to detect empty result sets, invalid SQL statements, and database connection failures.

The retrieved GeoDataFrame serves as the common data structure used by the visualization and export modules.

5.10 Administrative Boundary Visualization

To provide spatial context, GeoQuery automatically displays the administrative boundary corresponding to the user-specified location. After resolving the administrative level, the application retrieves the appropriate polygon from the States, Districts, or Subdistricts table and overlays it on the interactive map.

The boundary is displayed as a hollow polygon with a highlighted outline, allowing users to clearly visualize the extent of the selected administrative region without obscuring the retrieved spatial features.

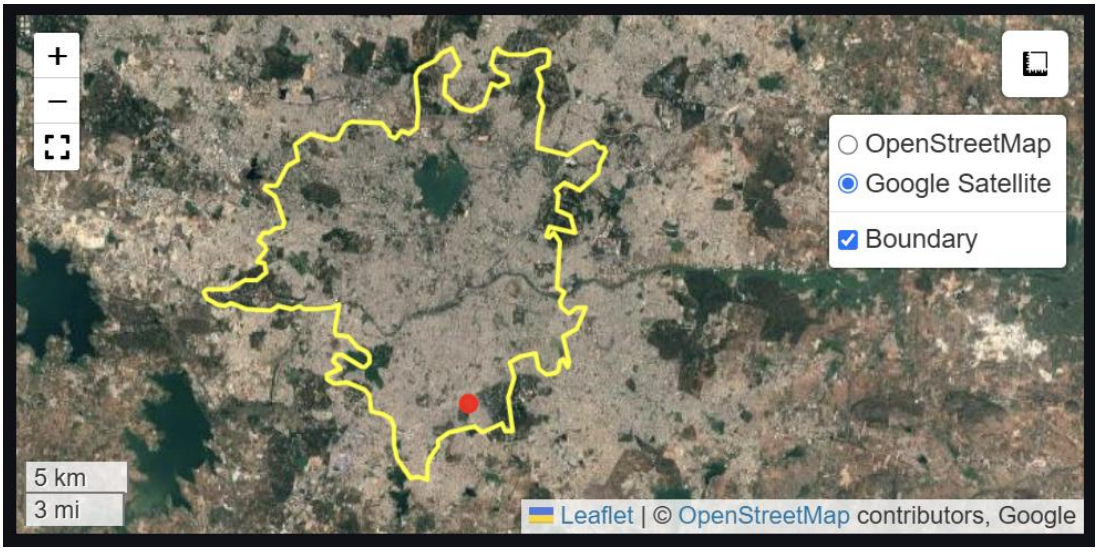

Figure 5.3: Administrative Boundary Visualization

5.11 Interactive Map Visualization

Spatial visualization is implemented using the Folium library, which is based on the Leaflet JavaScript mapping framework. The application supports multiple basemaps, including OpenStreetMap and Google Satellite imagery.

Depending on the geometry type, retrieved features are rendered using different visualization techniques. Point features are displayed as circular markers, while line and polygon features are rendered as GeoJSON layers. Automatic zooming is performed based on the spatial extent of the query results.

Additional map functionalities include fullscreen mode, measurement tools, layer controls, feature popups, and boundary highlighting.

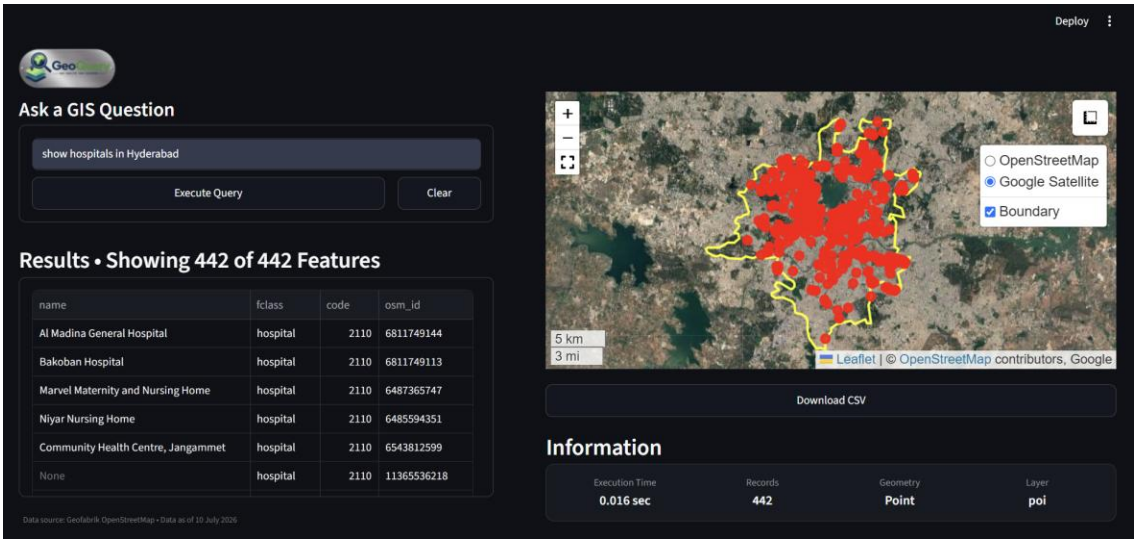

Figure 5.4: Interactive Map Display

5.12 Result Export

GeoQuery provides the option to export query results as CSV files. Prior to export, unnecessary geometry columns and internal database fields are removed to produce a clean attribute table suitable for further analysis in spreadsheet software or GIS applications.

Unlike the preview table shown within the application, the exported CSV file contains all retrieved records, ensuring that users receive the complete query results.

	A	B	C	D	E	F	G
109	Guru Nanak Care Hospital	hospital	2110	5118590649			
110	Sarojini Devi EYE Hospital	hospital	2110	11001534025			
111	eyes specalist	hospital	2110	13436972602			
112	Jagadamba Hospitals	hospital	2110	4416373792			
113	Tapadia Medical Centre's Eashwar Lakshmi Hospital	hospital	2110	2712662990			
114	Dental Care	hospital	2110	2779844594			
115	Sri Krishna	hospital	2110	6308287072			
116	Dangoria Maternity and Nursing Home	hospital	2110	6485066503			
117	Praja Sai Hospital	hospital	2110	6484888144			
118	ramesh eye care	hospital	2110	2712623077			
119	K Prakash Rao (eye specialist)	hospital	2110	2715310633			
120	Radha Rani Nursing Home	hospital	2110	2715310996			
121	Lions Club of Hyderabad Sadhuram Eye Hospital	hospital	2110	6480033798			
122	Centre For Sight - New Vision Laser Centers	hospital	2110	6496690608			
123	Hope Children's Hospital	hospital	2110	2192962541			
124	Dr. Mohan's Diabetes Specialities Centre	hospital	2110	6490784114			
125	Tarnaka Flyover;TGSRTC Hospital	hospital	2110	2583708951			
126	Anand Vitreo Retina Institute Pvt Ltd	hospital	2110	6486935781			
127	Suraksha Childrens Hospital	hospital	2110	6505613357			
128	PADMVATI MEDICAL CENTRE	hospital	2110	6549172458			
129	Innova Children's Heart Hospital	hospital	2110	665132280			
130	Apollo Clinic	hospital	2110	665132281			
131	hs	hospital	2110	10277927480			
132	Sri Santosh Hospital	hospital	2110	6497611600			
133	Manjary Multispeciality Hospital	hospital	2110	6496940849			
134	Community Health Centre, Lalapet	hospital	2110	10969330843			
135	SBR Pulse Hospital	hospital	2110	6549364392			

Figure 5.5: Exported results as csv

5.13 Application Workflow

The overall execution workflow of GeoQuery consists of the following sequential steps:

1. The user submits a natural language GIS query.
2. The query is interpreted by the Llama 3.3 language model.
3. The generated JSON is validated by the parser.
4. The location resolver determines the administrative level.
5. The SQL Query Builder constructs the corresponding spatial SQL query.
6. PostgreSQL/PostGIS executes the query.
7. GeoPandas retrieves the spatial results.
8. Administrative boundaries are loaded for visualization.
9. Folium displays the retrieved features on an interactive map.
10. Attribute information, execution statistics, and downloadable CSV results are presented to the user.

This modular workflow enables efficient integration of artificial intelligence with spatial database technologies while maintaining a clear separation between language understanding, spatial query generation, database execution, and visualization.

6 Results and Discussion

6.1 Overview

The GeoQuery application was successfully developed and tested using multiple natural language GIS queries on the prepared PostgreSQL/PostGIS spatial database. The system was evaluated to verify its ability to interpret user queries, generate appropriate spatial SQL statements, retrieve geographic information, and visualize the results on an interactive web map. Various categories of geographic features, including points of interest, waterways, transportation networks, natural features, and administrative boundaries, were queried to assess the overall functionality of the developed system.

The experimental results demonstrate that the integration of Large Language Models with spatial database technologies enables an intuitive and efficient method for accessing geospatial information. Users can retrieve geographic features without requiring knowledge of SQL or conventional GIS software, thereby significantly simplifying the interaction with spatial databases.

6.2 Application Results

The developed application successfully interprets user queries expressed in natural language and converts them into structured JSON representations, which are subsequently translated into executable spatial SQL queries. The generated queries are executed on the PostgreSQL/PostGIS database, and the retrieved spatial features are displayed on an interactive map together with their attribute information.

Several representative queries were executed during testing. Examples include retrieving hospitals within Hyderabad district, rivers in Telangana, railway stations in Andhra Pradesh, schools in specific administrative regions, and multiple feature classes such as hospitals and pharmacies. In each case, the system correctly identified the required thematic layer, administrative boundary, and feature class before executing the corresponding spatial query.

The application automatically determines whether the specified location corresponds to a state, district, or subdistrict and selects the appropriate administrative boundary for spatial filtering. The retrieved results are visualized with Folium, while attribute information is displayed in tabular form, along with execution statistics such as query execution time, geometry type, feature count, and dataset information.

The developed interface also enables users to export query results as CSV files, allowing further analysis with spreadsheet software or GIS applications.

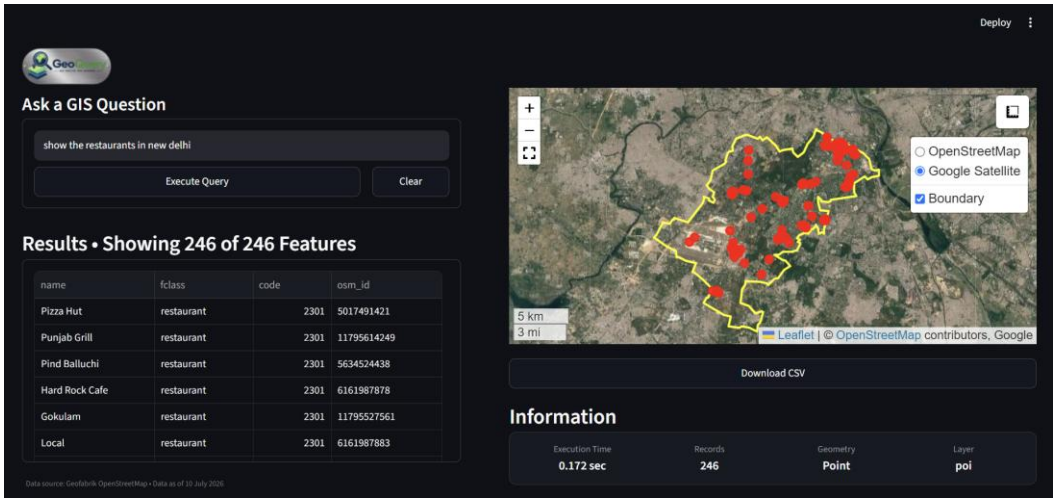

Figure 6.1: Restaurants in New Delhi

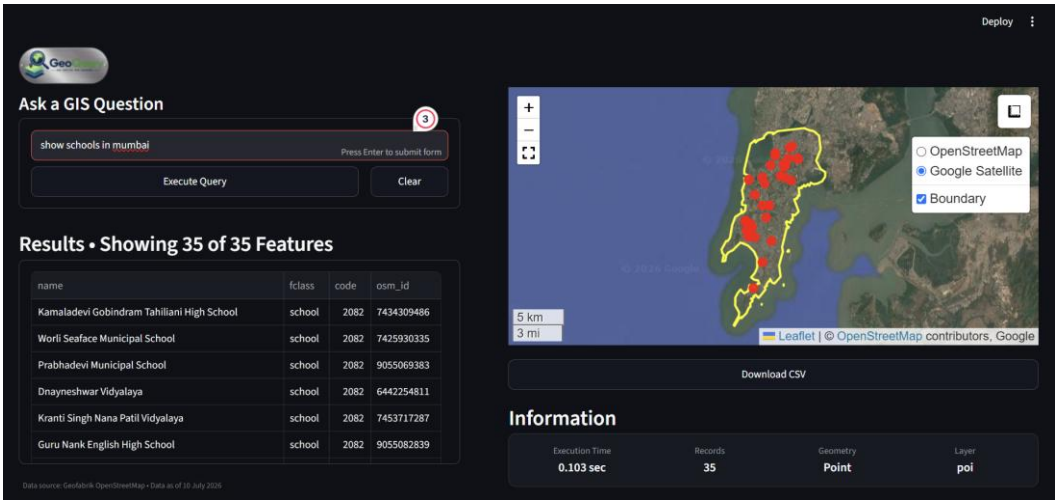

Figure 6.2: Schools in Mumbai

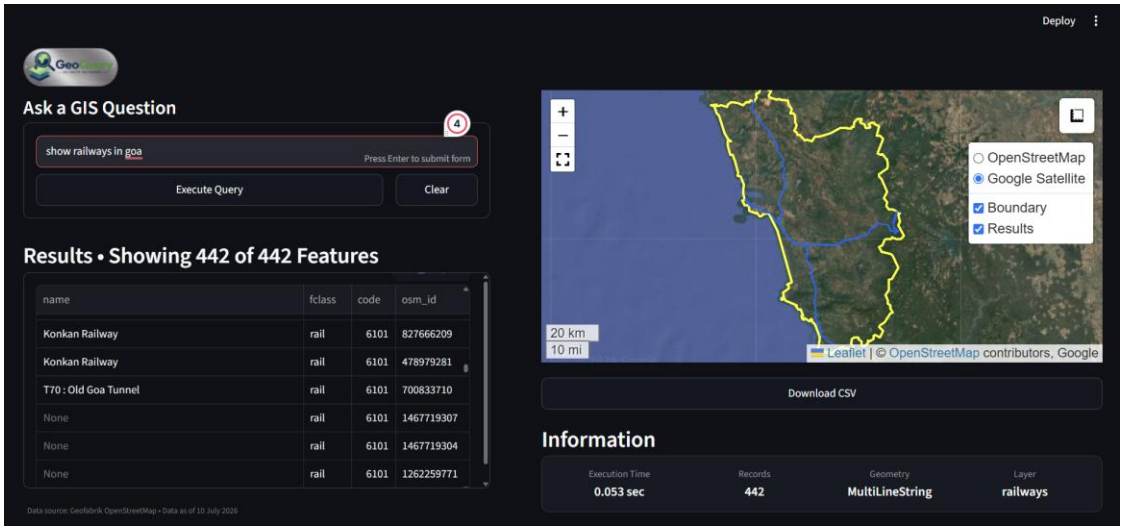

Figure 6.3: Railways in Goa

6.3 Discussion

The experimental results demonstrate that the GeoQuery system successfully integrates artificial intelligence with spatial database technologies to provide an intuitive natural language interface for geospatial information retrieval. The developed application enables users to retrieve geographic features by expressing their requirements in plain English, thereby eliminating the need to manually construct SQL queries or possess prior expertise in conventional GIS software. The modular architecture adopted during development effectively separates language understanding, query generation, database execution, and visualization, resulting in a system that is both flexible and maintainable.

One of the key strengths of the developed system is the use of a structured JSON representation as an intermediate step between the Large Language Model and the SQL Query Builder. Rather than allowing the language model to directly generate SQL statements, the planner extracts the user's intent and converts it into a structured format containing the required spatial layer, feature class, administrative location, and query constraints. The SQL Query Builder subsequently constructs executable spatial SQL queries based on these parameters. This approach improves system reliability, simplifies debugging, and minimizes the possibility of generating incorrect or unsafe SQL statements.

The integration of PostgreSQL with the PostGIS extension enables efficient execution of spatial queries using optimized spatial indexing and built-in geographic functions. Spatial predicates such as ST_Intersects() enable the application to accurately retrieve features within the specified administrative boundaries, while the location-resolution module automatically determines whether the user-specified location corresponds to a state, district, or subdistrict. This functionality significantly improves the system's usability by allowing users to specify locations naturally without requiring knowledge of administrative hierarchies.

The visualization module further enhances the application's usability by presenting the retrieved features on an interactive map, along with their corresponding administrative boundaries. The availability of multiple basemaps, automatic map zooming, feature popups, measurement tools, and downloadable query results provides a user-friendly environment for exploring spatial information. The generated attribute tables and execution statistics further assist users in understanding the retrieved data.

To assess the correctness of the implemented workflow, the retrieved spatial features were visually compared with the underlying OpenStreetMap basemap used within the application. Features such as hospitals, waterways, transportation networks, and administrative boundaries were observed to align with the corresponding locations and spatial extents shown in the OpenStreetMap reference data. The retrieved results consistently matched the underlying basemap, indicating that the generated spatial SQL queries successfully extracted the intended geographic features from the PostgreSQL/PostGIS database. This visual agreement provides confidence in the correctness of the implemented query planning, spatial filtering, database retrieval, and visualization processes.

Although the developed system successfully demonstrates the feasibility of natural language-based GIS querying, certain limitations remain. The current implementation primarily supports English-language queries and relies on the availability and completeness of OpenStreetMap data. Since natural language understanding is performed using the Groq-hosted Large Language Model, an active internet connection is required during query execution. Furthermore, the current version focuses on vector-based spatial information retrieval and does not support advanced raster analysis, network analysis, or complex geoprocessing operations. These limitations provide opportunities for future enhancements while not affecting the primary objective of simplifying geospatial information retrieval through natural language.

The experimental evaluation confirms that GeoQuery successfully achieves its intended objective of providing an intelligent, efficient, and user-friendly Natural Language GIS Query System. The integration of Large Language Models with PostgreSQL/PostGIS demonstrates the potential of conversational artificial intelligence to improve the accessibility of geospatial databases while preserving the robustness and analytical capabilities of modern GIS technologies.

6.4 Summary

The experimental evaluation confirms that GeoQuery successfully achieves the objectives of this project by enabling users to retrieve and visualize geospatial information through natural language. The system accurately interprets user intent, automatically resolves administrative boundaries, dynamically generates spatial SQL queries, executes them against a PostgreSQL/PostGIS database, and presents the results in an interactive mapping interface. The obtained results demonstrate that the proposed approach significantly improves the accessibility of spatial databases while maintaining the robustness and efficiency of conventional GIS systems.

7 Conclusion

This project presented the design and development of GeoQuery, an AI-powered Natural Language Geographic Information System that enables users to retrieve and visualize geospatial information using simple conversational queries. The system successfully integrates a Large Language Model with PostgreSQL/PostGIS to translate natural language into structured spatial queries without requiring users to possess prior knowledge of SQL or conventional GIS software. By combining language understanding, spatial databases, and interactive web-based visualization, the developed application demonstrates an effective approach for simplifying access to geographic information.

The primary objectives of the project were successfully achieved. A comprehensive spatial database covering the Indian subcontinent was prepared using OpenStreetMap data obtained from Geofabrik and organized within PostgreSQL/PostGIS. A modular application architecture was developed consisting of independent components for query planning, JSON parsing, location resolution, SQL query generation, database execution, and visualization. The system was capable of automatically identifying administrative boundaries at the state, district, and subdistrict levels and generating appropriate spatial SQL queries based on user requests. Experimental results confirmed that the retrieved spatial features were correctly visualized on interactive maps, and the displayed results matched the corresponding locations and extents visible on the underlying OpenStreetMap basemap, indicating the correctness of the implemented spatial querying workflow.

Although the developed system demonstrates the feasibility of natural language-based GIS querying, certain limitations remain. The current implementation primarily supports English-language queries and relies on the availability and completeness of OpenStreetMap data. Since the application uses a cloud-hosted Large Language Model through the Groq API, an active internet connection is required for query interpretation. Furthermore, the present system focuses mainly on the retrieval and visualization of vector datasets and supports a limited set of spatial operations. Advanced GIS functionalities such as raster analysis, network analysis, spatial statistics, terrain analysis, and complex geoprocessing workflows are not included in the current implementation.

The developed framework provides a strong foundation for future research and development. Future work may include support for multilingual natural language queries, integration of additional geospatial datasets from government agencies, and incorporation of advanced spatial analysis capabilities. The application can also be extended to support proximity searches, routing, shortest-path analysis, buffer generation, overlay analysis, and complex spatial relationships, such as containment, intersection, and adjacency, via conversational commands. Further improvements may involve integrating Google Earth Engine for remote sensing analysis, supporting raster datasets, enabling voice-based interaction, and deploying the application as a cloud-based GIS service for broader public use.

Future versions may integrate Retrieval-Augmented Generation (RAG) with geospatial

documents and support multimodal interaction using text, voice, and maps.

In conclusion, the GeoQuery system demonstrates `that integrating Large Language Models with spatial database technologies provides an intuitive, efficient, and user-friendly approach to geospatial information retrieval. The project illustrates the potential of conversational artificial intelligence to overcome the technical barriers of traditional GIS software while preserving the analytical capabilities of modern spatial databases. The developed application serves as a practical example of how recent advances in artificial intelligence can improve the accessibility of geospatial technologies for researchers, planners, students, and general users.

8 References

- PostgreSQL Global Development Group. (2025). *PostgreSQL Documentation*. https://www.postgresql.org/docs/
- PostGIS Project Steering Committee. (2025). *PostGIS Documentation*. https://postgis.net/documentation/
- OpenStreetMap Contributors. (2025). *OpenStreetMap*. https://www.openstreetmap.org
- Geofabrik GmbH. (2025). *OpenStreetMap Data Extracts*. https://download.geofabrik.de/
- Streamlit Inc. (2025). *Streamlit Documentation*. https://docs.streamlit.io/
- Jordahl, K., et al. (2024). *GeoPandas: Python tools for geographic data*. https://geopandas.org/
- Python Visualization Folium. (2025). *Folium Documentation*. https://python-visualization.github.io/folium/
- Bayer, M. (2025). *SQLAlchemy Documentation*. https://www.sqlalchemy.org/
- Groq Inc. (2025). *Groq API Documentation*. https://console.groq.com/docs
- Dubey, A., et al. (2024). *The Llama 3 Herd of Models*. Meta AI.
- Dubey, A., Abhinav, et al. The Llama 3 Herd of Models. Meta AI, 2024. https://arxiv.org/abs/2407.21783